Programmer efficacement avec des patrons de conception

Qu'est-ce qu'un patron de conception ?

Un patron de conception est une solution éprouvée, réutilisable et nommée à un problème récurrent de conception logicielle.
Il décrit une organisation de classes/objets (statique ou dynamique) que l'on adapte ensuite au contexte, comme l'exemple d'une roue de secours : même idée générale, implémentations variées.

Les patrons sont décrits selon quatre éléments essentiels :

- Nom : donne un vocabulaire commun.
- Problème : à quel contexte le patron s'applique.
- Solution : structure générale (classes, rôles, interactions).
- Conséquences : effets positifs et limites.

Pourquoi les patrons existent ?

Dans les années 1990, les quatre auteurs du Gang of Four (GoF) ont analysé des centaines de projets et remarqué que les mêmes problèmes revenaient, avec souvent les mêmes solutions.
Ils ont rassemblé ces solutions dans Design Patterns: Elements of Reusable Object-Oriented Software (1994) et les ont classées en trois familles :

- Patrons de création : comment instancier des objets.
- Patrons structurels : comment organiser et assembler les classes.
- Patrons comportementaux : comment les objets collaborent.

Identifier le bon patron

La partie la plus difficile n'est pas l'implémentation, mais reconnaître le problème.
Exemple : dans un jeu de cartes, générer différents tas avec des configurations variées correspond à un problème de création d'objets → donc à un patron de création.

Un patron ne décrit pas la fabrication d'un jeu de cartes en particulier, mais une manière générale de produire des objets.
On l'adapte ensuite pour le cas concret.

Conséquences de l'utilisation d'un patron

Chaque patron apporte :

- au moins une conséquence positive (structure claire, solution solide) ;
- parfois des limites (ex. l'espace perdu par la roue de secours dans un coffre).

Pourquoi utiliser un patron de conception ?

Bénéfices principaux

- Fiabilité : la solution a déjà fait ses preuves.
- Lisibilité : les patrons sont documentés et connus.
- Communication facilitée : dire « utilise un Factory Method » suffit.
- Qualité du code : évite les bricolages difficiles à maintenir.

En résumé

- Les patrons offrent des solutions réutilisables à des problèmes courants.
- Ils reposent sur 4 éléments : nom, problème, solution, conséquences.
- Ils améliorent compréhension, maintenance et communication.
- Ils se répartissent en 3 catégories : création, structure, comportement.

Créer des objets avec les patrons de création

Pourquoi utiliser des patrons de création ?

Pour qu'un programme fonctionne, il doit instancier des objets.
Le réflexe typique est d'écrire :

```
new someClassName();
```

Mais cette approche pose un problème :

➡️ le type instancié est figé dans le code.
Pour en changer, il faut revenir modifier le code existant, ce qui augmente le risque d'erreurs et réduit la flexibilité.

Une bonne pratique consiste à éviter autant que possible l'utilisation directe de new.
On cherche à décaler la responsabilité de créer les objets vers un autre composant.
C'est ce « niveau d'indirection » qui permet plus de souplesse.

C'est exactement le rôle des patrons de création : offrir un moyen flexible et contrôlé de fabriquer des objets.

Le Factory Pattern (patron d'usine)

Le Factory Pattern est l'un des patrons de création les plus simples.
Son principe est :

➡️ un objet dédié (la "factory") se charge de créer d'autres objets.
Le code client ne connaît pas les détails de construction : il demande simplement un objet à l'usine.

Pourquoi une factory dans un jeu de cartes ?

Supposons que certains jeux utilisent 32 cartes au lieu de 52.

Si le deck est directement créé dans le contrôleur (ex. new Deck()), changer de type de deck implique de modifier ce contrôleur.

En déplaçant la création dans une classe factory, on gagne :

- flexibilité : choisir dynamiquement le type de deck ;
- maintenabilité : aucun changement dans le contrôleur ;
- testabilité : créer facilement des decks de test.

Exemple d'utilisation

On peut définir plusieurs deck :

- NormalDeck : crée un deck de 32 cartes
- SmallDeck : crée un deck standard
- TestDeck : crée un deck spécial pour les tests

Le contrôleur ne fait plus :

```
new Deck();
```

mais :

```
Deck deck = deckFactory.makeDeck(Small);
```

Ainsi, changer de type de deck consiste simplement à changer d'usine, pas à modifier le code métier.

En résumé

- Les patrons de création évitent de dépendre directement de ne.
- La factory centralise la création des objets.
- Changer ce qui est créé se fait en changeant la factory, pas le code client.
- C'est un moyen simple d'améliorer la flexibilité et la maintenabilité.

```
1  public class DeckFactory {
2      public static Deck makeDeck(DeckType type) {
3          switch (type) {
4              case Normal: return new NormalDeck();
5              case Small: return new SmallDeck();
6              case Test: return new TestDeck();
7          }
8
9          // fallback
10         return new NormalDeck();
11     }
12 }
```

Ensuite, n'importe quelle classe ayant besoin d'un deck normal pourrait appeler :

```
1  Deck myDeck = DeckFactory.makeDeck(DeckType.Normal);
```

Flexibilité accrue grâce à la factory

Adapter la création des decks

Nous avons gagné en flexibilité en déléguant la création des decks à un objet dédié (DeckFactory).
Selon les besoins, celle-ci peut produire :

- NormalDeck
- SmallDeck
- TestDeck

La factory peut également être adaptée :
on peut lui ajouter une méthode de configuration permettant de choisir à l'avance quel type de deck elle créera.
Dans ce cas, la méthode makeDeck() n'aurait plus de paramètre : elle retournerait simplement le deck correspondant à la configuration précédente.

Cette approche est particulièrement utile pour les tests unitaires :

- en test, la factory est configurée pour produire un TestDeck ;
- pendant un vrai jeu, elle produit un NormalDeck ou un SmallDeck.

Une flexibilité dynamique

Le Factory Pattern est un patron utilisé au moment de l'exécution, lorsqu'il faut pouvoir choisir facilement entre plusieurs implémentations.

L'idée est simple :
➡️ on indique à la factory ce que l'on veut, et elle créé l'objet approprié.

Cela permet de modifier le type d'objet généré sans changer le code client.

Le Builder Pattern

Principe général

Le Builder Pattern sert à construire un objet complexe en assemblant plusieurs éléments, dont le choix peut varier.
L'idée centrale est que :

- l'objet final est toujours composé des mêmes types de parties ;
- mais chaque partie peut être sélectionnée parmi plusieurs variantes ;
- le builder suit un algorithme de construction qui décide quelles pièces créer et comment les assembler.

Un exemple concret : le menu de fast-food

Commander un menu dans un fast-food illustre parfaitement ce patron :

- un élément principal : taco, hamburger, crêpes…
- un accompagnement : haricots, frites, pain grillé…
- une boisson : soda, café, thé…

Le résultat est toujours un menu complet (3 éléments),
mais les variantes choisies peuvent changer à chaque commande.

Le builder fonctionne exactement de cette façon :
il construit un objet complexe en sélectionnant les bonnes variantes de chaque partie.

Application possible au jeu de cartes

Dans notre jeu, le Builder Pattern pourrait être utilisé (même s'il peut sembler un peu lourd).
Le joueur ou la configuration du jeu peut déterminer :

- la taille du deck (32 cartes, 52 cartes, etc.)
- les règles de victoire (carte haute, carte basse…)

Le builder recevrait ces choix et renverrait :

➡️ un objet Deck correctement configuré,
➡️ ainsi que la bonne implémentation des règles,
en fonction des options sélectionnées.

```java
public interface GameBuilder {
    Game getGame();
}

public class NormalHighCardGameBuilder implements GameBuilder {
    public Game getGame() {
        return new Game(DeckFactory.makeDeck(DeckType.Normal),
                        EvaluatorFactory.makeEvaluator(EvaluatorType.High));
    }
}

public class SmallHighCardGameBuilder implements GameBuilder {
    public Game getGame() {
        return new Game(DeckFactory.makeDeck(DeckType.Small),
                        EvaluatorFactory.makeEvaluator(EvaluatorType.High));
    }
}
```

Organiser des objets avec des patrons de conception structurelle

Que sont les patrons de conception structurelle?

Lorsque vous faites vos courses, vous apportez vos articles en caisse et payez. Vous interagissez avec le caissier. C'est assez simple. C'est un exemple de **patron structurel où une grande quantité de complexité est cachée** derrière une interface simple. En tant que client, vous ne voulez pas avoir à commander des produits, à les expédier, à les tarifer ou à les mettre sur les étagères. Vous voulez faire du shopping et rentrer chez vous.

Les patrons structurels sont autant de façons d'**organiser les classes ou les objets** afin qu'ils soient **faciles à utiliser.** Il peut se passer beaucoup de choses dans les coulisses, mais en tant qu'utilisateur, vous n'avez pas besoin de connaître les complexités. Il existe deux types de patrons structurels : ceux qui organisent les classes et ceux qui organisent les objets. Nous allons examiner les deux types.

Qu'est-ce que le patron composite?

Il y a des moments où vous voulez traiter un tas d'objets et des objets uniques de la même manière. Par exemple, dessiner une forme. Si vous avez une seule forme, disons un cercle, vous aimeriez appeler:

```
1 Shape shape = new Circle();
2 shape.draw();
```

Mais vous aimeriez aussi appeler toute une collection de formes, de la même manière:

```
1 Shape shapes = new ComplicatedDiagramOfABunchOfShapes();
2 shapes.draw();
```

C'est ce que fait le patron de conception composite. Chaque élément, qu'il s'agisse d'une entité unique ou d'une collection, expose la même interface. L'implémentation de la collection parcourt tous les objets qu'elle possède, en demandant à chacun d'exécuter la méthode demandée. Dans le chapitre suivant, nous ajouterons plusieurs vues avec un autre patron. En prévision de cela, nous voulons que le contrôleur de jeu traite une vue, ou plusieurs, comme si elles étaient identiques.

Nous allons placer toute cette gestion des vues dans une nouvelle classe composite :
Nous avons remplacé le fait de ne montrer qu'une seule vue, par une liste de vues. Ensuite, chaque fois que nous demandons à une vue de mettre à jour son affichage, nous parcourons la liste des vues, demandant à chacune la même chose. Mais vous remarquerez que nous n'avons pas à changer les noms des méthodes pour rendre cette fonctionnalité plus flexible. Une seule ou plusieurs vues, au regard des appelants de la fonction showPlayerName, n’a aucune incidence.



```java
public class GameViewables implements GameViewable {

  public ArrayList<GameViewable> gameViewables;

  public GameViewables() {
    this.gameViewables = new ArrayList<>();
  }

  public void addGameViewable(GameViewable gv) {
    gameViewables.add(e: gv);
  }

  @Override
  public void promptForFlip() {
    for (GameViewable gv : gameViewables) {
      gv.promptForFlip();
    }

  }

  @Override
  public void promptForNewGame() {
    for (GameViewable gv : gameViewables) {
      gv.promptForNewGame();
    }

  }

  @Override
  public void promptForPlayerName() {
    for (GameViewable gv : gameViewables) {
      gv.promptForPlayerName();
    }

  }

  @Override
  public void setController(GameController gc) {
    for (GameViewable gv : gameViewables) {
      gv.setController(gc);
    }

  }
```

Gérer des objets avec des patrons de conception comportementale

Patrons de conception comportementaux

Les patrons de conception comportementaux décrivent comment les objets collaborent entre eux : qui parle à qui, comment les décisions se prennent, comment les messages circulent dans un système.

Pour s'en faire une idée, on peut regarder ce qui se passe dans un grand aéroport. Des dizaines d'avions décollent et atterrissent en même temps. Une mauvaise solution serait que chaque pilote discute directement avec tous les autres pour se coordonner : au-delà de quelques avions, ce serait totalement ingérable.

La solution réelle, c'est la tour de contrôle : un acteur central qui coordonne les mouvements de tous les avions. Les pilotes ne se parlent pas entre eux, ils s'adressent à un médiateur (la tour), qui prend les décisions et renvoie les instructions.
En génie logiciel, ce type d'organisation est capturé par le patron Médiateur : un objet central gère les interactions entre plusieurs objets qui, eux, ne se connaissent pas directement.

De manière générale, les patrons comportementaux proposent des schémas de communication réutilisables pour éviter le chaos quand de nombreux objets doivent interagir.

Le patron Observateur

Un autre patron comportemental classique est le patron Observateur.
Il intervient lorsqu'un objet détient un état (des données, une progression, un résultat…) dont plusieurs autres objets doivent être informés.

Un exemple simple : la télévision.
La chaîne diffuse un programme, que vous décidiez de regarder ou non. Les spectateurs peuvent arriver, partir, changer de chaîne ; la télé, elle, ne change rien à sa façon de diffuser. Elle « émet » un flux, et chacun est libre de s'y abonner ou de s'en désabonner.

C'est exactement l'idée du patron Observateur :

- un sujet diffuse des informations (par exemple, l'état d'un jeu) ;
- des observateurs peuvent s'inscrire et se désinscrire librement ;
- le sujet n'a pas besoin de changer quand on ajoute ou enlève des observateurs, il continue simplement à notifier ceux qui sont inscrits.

Application à notre jeu de cartes

Dans notre cas, le modèle du jeu (la partie, les joueurs, les cartes) joue le rôle de sujet : c'est lui qui porte l'état.
Les différentes vues (GameViewable) jouent le rôle d'observateurs : elles doivent être informées quand quelque chose change dans le jeu (nouveau joueur, cartes distribuées, gagnant, etc.).

On aimerait donc pouvoir :

- afficher le même jeu sur plusieurs vues ;
- et surtout, ajouter de nouvelles vues à tout moment, sans modifier la logique du jeu.

Plutôt que de câbler en dur un nombre fixe de vues, nous allons considérer les GameViewable comme des observateurs que l'on peut créer et enregistrer dynamiquement.

Travail demandé :
pour préparer cela, vous allez ajouter un nouveau bouton dans la vue Swing qui, à chaque clic, crée un nouvel objet GameViewable (par exemple une nouvelle instance de GameSwing) et l'enregistre auprès du contrôleur comme nouvelle « vue » du jeu.
Implémenter la méthode controller.addView(newView) de sorte que la vue soit ajoutée à la liste de vues de gamesViewables et que ces vues soient "informées" (setController) du contrôleur déjà présent.

```java
// when clicked, tell controller to add a new view and set its controller
private void addAddGameViewButton(Container contentPane) {
  btnAddGameView = new JButton(text: "Add View");
  addCenteredComponent(component: btnAddGameView, contentPane);
  btnAddGameView.addActionListener(l: new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
      textArea.append(str: "Cards dealt:\n");
      GameSwing newView = new GameSwing();
      newView.createAndShowGUI();
      controller.addView(newView);
    }
  });
}
```

Gestion des thread et de la concurrence entre les vues

Nous allons travailler sur l'implémentation de cette nouvelle classe permettant de manière concurrente la mise à jour des nouvelles vues créées.

Coordination de plusieurs vues : limites de la boucle infinie de la machine à état.
Dans la version en ligne de commande, le contrôleur pilotait le jeu via une méthode :

```java
public void run() {
  while (true) {
    switch (gameState) {
      case AddingPlayer -> view.promptForPlayerName();
      case CardsDealt -> view.promptForFlip();
      case WinnerRevealed -> view.promptForNewGame();
    }
  }
}
```

Cette boucle infinie est acceptable en CLI : le programme attend des entrées utilisateur bloquantes, et la boucle tourne « au rythme » des Scanner et des System.in.read().

Passage à plusieurs vues avec GameViewables

Quand on introduit un composite de vues (Cf chapitre "composite") et qu'on permet d'ajouter dynamiquement des vues via un bouton dans la vue Swing, on se retrouve avec deux mondes qui se marchent dessus :

- d'un côté, la boucle infinie run() qui parcourt en permanence la liste des vues (GameViewables) ;

- de l'autre, le thread graphique Swing (Event Dispatch Thread) qui, en réponse à un clic, appelle addGameViewable(newView) pour ajouter une nouvelle vue dans cette même liste.

Résultat : on ouvre la porte à une condition de course (race condition) et à des erreurs de type ConcurrentModificationException : la liste est modifiée par un thread pendant qu'un autre est en train de l'itérer.

En plus, la boucle infinie monopolise un cœur CPU et n'est pas compatible avec le modèle évènementiel de Swing : ce n'est plus le contrôleur qui doit « tourner tout seul », mais l'interface qui déclenche des actions à travers des callbacks.

Sortir du modèle CLI : de run() à runOneStep()

Pour s'affranchir définitivement de ce fonctionnement hérité de la CLI, on remplace la boucle infinie par une méthode qui ne fait qu'une seule étape de la machine à états :
Ici, plus de while (true) :
runOneStep() regarde simplement l'état courant (gameState) et demande à toutes les vues du composite GameViewables d'afficher la consigne correspondante.

```
public GameController(Game game, GameViewable _gameViewable) {
  this.deck = game.getDeck();
  this.gameViewables = new GameViewables();
  gameState = GameState.AddingPlayer;
  players = new ArrayList<>();
  this._gameEvaluator = game.getGe();
  this.addView(newView: _gameViewable);
}

public void runOneStep() {
  switch (gameState) {
    case AddingPlayer -> gameViewables.promptForPlayerName();
    case CardsDealt -> gameViewables.promptForFlip();
    case WinnerRevealed -> gameViewables.promptForNewGame();
  }
}
```

Appel initial depuis le main

Au démarrage, le main crée la première vue, puis le contrôleur, puis lance une seule étape :

À ce stade, l'état initial du jeu est généralement AddingPlayer : les vues affichent donc la consigne liée à l'ajout de joueurs.

```java
package com.iut.cards;

import com.iut.cards.builder.Game;
import com.iut.cards.builder.HighCardSmallDeck;
import com.iut.cards.controller.GameController;
import com.iut.cards.view.GameSwing;

public class App {
  public static void main(String[] args) {
    GameSwing gs = new GameSwing();
    gs.createAndShowGUI();
    Game game = new HighCardSmallDeck().getGame();
    GameController gc = new GameController(game, _gameViewable: gs);
    gc.runOneStep();
  }
}
```

Relancer la machine à états depuis les méthodes du contrôleur

Comme il n'y a plus de boucle infinie, ce sont désormais les méthodes du contrôleur qui relancent explicitement une étape de la machine à états à chaque fois que le modèle ou l'état du jeu change.

Par exemple :

```java
public void addPlayer(String name) {
  Player play = new Player(name);
  players.add(e: play);
  gameViewables.showPlayerName(index: players.indexOf(o: play), name: play.getName());
  runOneStep();
}

public void startGame() {
  deck.shuffle();
  if (gameState != GameState.CardsDealt) {
    for (Player p : players) {
      p.addCard(card: deck.removeTopCard());
      gameViewables.showFaceDownCardForPlayer(index: players.indexOf(o: p), name: p.getName());
    }
  }
  gameState = GameState.CardsDealt;
  runOneStep();
}
```

Le schéma devient :

1. Une vue (Swing) déclenche une action → bouton, menu, etc.
2. Le contrôleur met à jour le modèle (joueurs, cartes, deck, état du jeu).
3. Le contrôleur appelle runOneStep() pour refléter le nouvel état dans toutes les vues.